

Project 01: Library Management System

Difficulty: Beginner | Estimated Time: 1 Week

1. Project Overview and Goals

The Library Management System (LMS) is a foundational database project that models the core operations of a public or academic library. You will design and implement a relational database that handles book cataloging, member registration, borrowing and returning transactions, fines, and basic reporting.

Goals:

- Practice designing a normalized relational schema from a real-world domain.
 - Write CRUD queries across multiple related tables.
 - Implement business rules using constraints, triggers, and stored procedures.
 - Generate reports using aggregations, JOINS, and window functions.
-

2. Learning Objectives

By completing this project, you will be able to:

- Design a multi-table schema with proper primary keys, foreign keys, and constraints.
 - Use `CHECK`, `NOT NULL`, `UNIQUE`, and `DEFAULT` constraints effectively.
 - Write `JOIN` queries across 3-4 tables.
 - Use aggregate functions (`COUNT`, `SUM`, `AVG`) with `GROUP BY` and `HAVING`.
 - Implement PL/pgSQL stored functions for business logic.
 - Write triggers that automatically enforce rules on data changes.
 - Create indexes to support common query patterns.
 - Use `SERIAL` / `IDENTITY` columns and sequences.
-

3. Functional Requirements

- **Books:** Store title, ISBN, author(s), genre, publisher, publication year, total copies, and available copies.
 - **Members:** Register members with name, email, phone, address, membership type, and join date.
 - **Borrowing:** Track which member borrowed which book copy, when, and when it was returned.
 - **Fines:** Calculate and record fines when books are returned late.
 - **Reservations:** Allow members to reserve books that are currently checked out.
 - **Staff:** Track which staff member processed each transaction.
 - **Reports:** Active loans, overdue books, popular titles, member borrowing history.
-

4. Non-Functional Requirements

- ISBN must be unique and validated (10 or 13 digits).
- A member cannot borrow more than 5 books at a time.
- The loan period is 14 days; fine rate is \$0.25/day overdue.
- Returning a book automatically decrements `available_copies` on checkout and increments on return.
- All monetary values stored as `NUMERIC(10,2)`.

- Timestamps stored with timezone (`TIMESTAMPTZ`).

5. Complete Database Schema

```
-- =====
-- LIBRARY MANAGEMENT SYSTEM - Complete Schema
-- PostgreSQL 15+
-- =====

CREATE SCHEMA IF NOT EXISTS library;
SET search_path = library, public;

-- -----
-- PUBLISHERS
-- -----

CREATE TABLE publishers (
    publisher_id SERIAL PRIMARY KEY,
    name         VARCHAR(200) NOT NULL,
    country      VARCHAR(100),
    website      VARCHAR(300),
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE publishers IS 'Book publishers catalog';

-- -----
-- AUTHORS
-- -----

CREATE TABLE authors (
    author_id     SERIAL PRIMARY KEY,
    first_name    VARCHAR(100) NOT NULL,
    last_name     VARCHAR(100) NOT NULL,
    birth_year    SMALLINT CHECK (birth_year BETWEEN 1000 AND 2100),
    nationality    VARCHAR(100),
    bio           TEXT,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE authors IS 'Book authors';
CREATE INDEX idx_authors_name ON authors (last_name, first_name);

-- -----
-- GENRES
-- -----

CREATE TABLE genres (
    genre_id     SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL UNIQUE,
    parent_id    INTEGER REFERENCES genres(genre_id)
);
```

```
-- BOOKS
```

```
CREATE TABLE books (  
    book_id SERIAL PRIMARY KEY,  
    isbn VARCHAR(13) NOT NULL UNIQUE,  
    title VARCHAR(400) NOT NULL,  
    publisher_id INTEGER NOT NULL REFERENCES publishers(publisher_id),  
    genre_id INTEGER REFERENCES genres(genre_id),  
    publication_year SMALLINT CHECK (publication_year BETWEEN 1400 AND 2100),  
    language VARCHAR(50) NOT NULL DEFAULT 'English',  
    total_copies SMALLINT NOT NULL DEFAULT 1 CHECK (total_copies >= 0),  
    available_copies SMALLINT NOT NULL DEFAULT 1 CHECK (available_copies >= 0),  
    location_code VARCHAR(20), -- shelf/aisle reference  
    description TEXT,  
    cover_image_url VARCHAR(500),  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    CONSTRAINT chk_copies CHECK (available_copies <= total_copies)  
);
```

```
COMMENT ON TABLE books IS 'Central book catalog';  
COMMENT ON COLUMN books.isbn IS 'ISBN-10 or ISBN-13, stored without hyphens';  
COMMENT ON COLUMN books.location_code IS 'Physical shelf location, e.g. A-12-3';
```

```
CREATE INDEX idx_books_title ON books USING GIN (to_tsvector('english',  
title));  
CREATE INDEX idx_books_publisher ON books (publisher_id);  
CREATE INDEX idx_books_genre ON books (genre_id);  
CREATE INDEX idx_books_available ON books (available_copies) WHERE available_copies  
> 0;
```

```
-- BOOK_AUTHORS (many-to-many)
```

```
CREATE TABLE book_authors (  
    book_id INTEGER NOT NULL REFERENCES books(book_id) ON DELETE CASCADE,  
    author_id INTEGER NOT NULL REFERENCES authors(author_id),  
    role VARCHAR(50) NOT NULL DEFAULT 'Author'  
    CHECK (role IN ('Author', 'Co-  
Author', 'Editor', 'Translator', 'Illustrator')),  
    PRIMARY KEY (book_id, author_id)  
);
```

```
-- MEMBERSHIP TYPES
```

```
CREATE TABLE membership_types (  
    type_id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL UNIQUE,  
    max_books SMALLINT NOT NULL DEFAULT 3 CHECK (max_books > 0),  
    loan_days SMALLINT NOT NULL DEFAULT 14 CHECK (loan_days > 0),
```

```

        annual_fee    NUMERIC(8,2) NOT NULL DEFAULT 0.00,
        description    TEXT
    );

INSERT INTO membership_types (name, max_books, loan_days, annual_fee, description)
VALUES
    ('Basic',    3,  14,    0.00, 'Free membership for residents'),
    ('Standard', 5,  21,   25.00, 'Standard paid membership'),
    ('Premium',  10, 30,   50.00, 'Premium membership with extended privileges'),
    ('Student',  5,  21,    0.00, 'Free for enrolled students');

-----
-- MEMBERS
-----

CREATE TABLE members (
    member_id        SERIAL PRIMARY KEY,
    first_name       VARCHAR(100) NOT NULL,
    last_name        VARCHAR(100) NOT NULL,
    email            VARCHAR(200) NOT NULL UNIQUE,
    phone            VARCHAR(20),
    address          TEXT,
    date_of_birth    DATE,
    type_id          INTEGER NOT NULL REFERENCES membership_types(type_id) DEFAULT 1,
    membership_date  DATE NOT NULL DEFAULT CURRENT_DATE,
    expiry_date      DATE NOT NULL DEFAULT (CURRENT_DATE + INTERVAL '1 year')::DATE,
    is_active        BOOLEAN NOT NULL DEFAULT TRUE,
    total_fines      NUMERIC(10,2) NOT NULL DEFAULT 0.00,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_email_format CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
);

COMMENT ON TABLE members IS 'Library members / patrons';
CREATE INDEX idx_members_email    ON members (email);
CREATE INDEX idx_members_name     ON members (last_name, first_name);
CREATE INDEX idx_members_active   ON members (is_active) WHERE is_active = TRUE;

-----
-- STAFF
-----

CREATE TABLE staff (
    staff_id        SERIAL PRIMARY KEY,
    first_name      VARCHAR(100) NOT NULL,
    last_name       VARCHAR(100) NOT NULL,
    email           VARCHAR(200) NOT NULL UNIQUE,
    role            VARCHAR(50) NOT NULL DEFAULT 'Librarian'
                    CHECK (role IN ('Librarian', 'Senior Librarian', 'Admin', 'Manager')),
    is_active       BOOLEAN NOT NULL DEFAULT TRUE,
    hired_at        DATE NOT NULL DEFAULT CURRENT_DATE
);
-----

```

-- LOANS

```
-----  
CREATE TABLE loans (  
    loan_id        SERIAL PRIMARY KEY,  
    member_id      INTEGER NOT NULL REFERENCES members(member_id),  
    book_id        INTEGER NOT NULL REFERENCES books(book_id),  
    staff_id       INTEGER REFERENCES staff(staff_id),  
    loan_date      TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    due_date       DATE NOT NULL,  
    return_date    TIMESTAMPTZ,  
    status         VARCHAR(20) NOT NULL DEFAULT 'Active'  
                CHECK (status IN ('Active', 'Returned', 'Overdue', 'Lost')),  
    notes          TEXT  
);  
  
COMMENT ON TABLE loans IS 'Book checkout and return transactions';  
CREATE INDEX idx_loans_member    ON loans (member_id);  
CREATE INDEX idx_loans_book      ON loans (book_id);  
CREATE INDEX idx_loans_status    ON loans (status) WHERE status IN  
( 'Active', 'Overdue' );  
CREATE INDEX idx_loans_due_date  ON loans (due_date) WHERE return_date IS NULL;
```

-- FINES

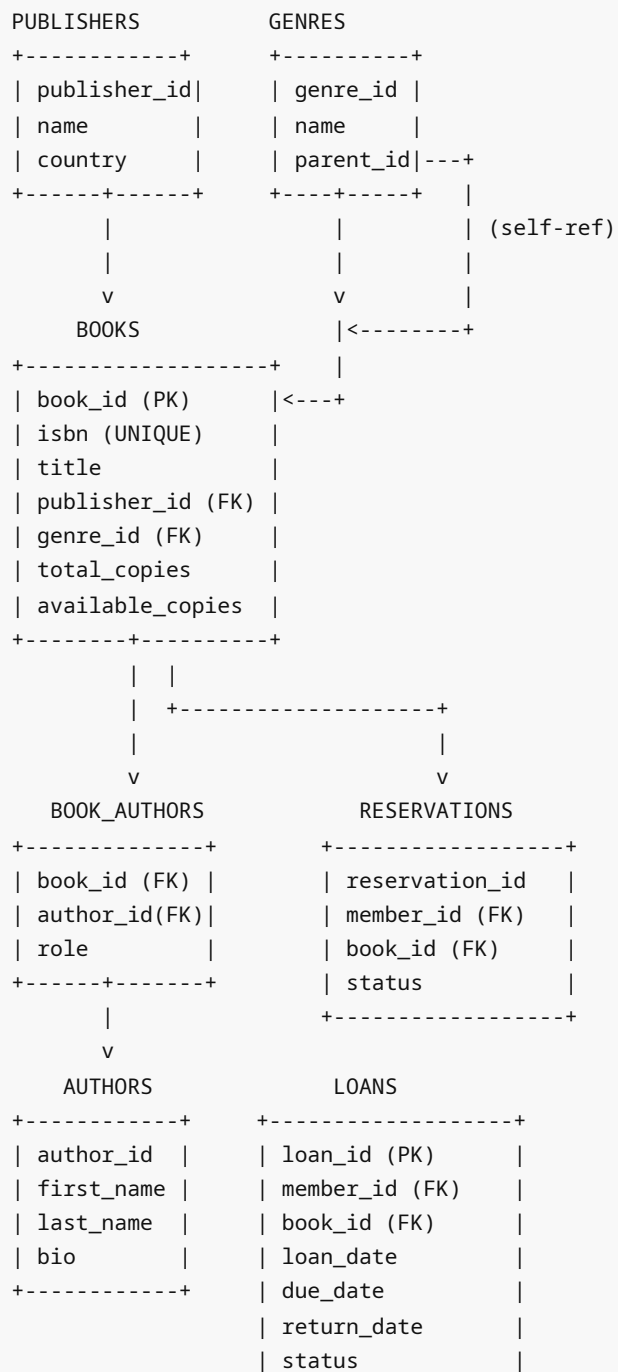
```
-----  
CREATE TABLE fines (  
    fine_id        SERIAL PRIMARY KEY,  
    loan_id        INTEGER NOT NULL UNIQUE REFERENCES loans(loan_id),  
    member_id      INTEGER NOT NULL REFERENCES members(member_id),  
    amount         NUMERIC(10,2) NOT NULL CHECK (amount > 0),  
    reason         VARCHAR(100) NOT NULL DEFAULT 'Late Return',  
    issued_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    paid_at        TIMESTAMPTZ,  
    is_paid        BOOLEAN NOT NULL DEFAULT FALSE  
);  
  
CREATE INDEX idx_fines_member    ON fines (member_id);  
CREATE INDEX idx_fines_unpaid    ON fines (member_id) WHERE is_paid = FALSE;
```

-- RESERVATIONS

```
-----  
CREATE TABLE reservations (  
    reservation_id SERIAL PRIMARY KEY,  
    member_id      INTEGER NOT NULL REFERENCES members(member_id),  
    book_id        INTEGER NOT NULL REFERENCES books(book_id),  
    reserved_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    expires_at     TIMESTAMPTZ NOT NULL DEFAULT (NOW() + INTERVAL '7 days'),  
    status         VARCHAR(20) NOT NULL DEFAULT 'Pending'  
                CHECK (status IN  
( 'Pending', 'Ready', 'Fulfilled', 'Expired', 'Cancelled' )),  
    notified_at    TIMESTAMPTZ
```

```
CREATE INDEX idx_reservations_member ON reservations (member_id);
CREATE INDEX idx_reservations_book ON reservations (book_id);
CREATE UNIQUE INDEX idx_reservations_active
ON reservations (member_id, book_id)
WHERE status IN ('Pending', 'Ready');
```

6. ASCII ER Diagram



```

MEMBERSHIP_TYPES +-----+-----+
+-----+
| type_id (PK) |          v
| name         |          FINES
| max_books    | +-----+
| loan_days    | | fine_id (PK) |
+-----+---+ | loan_id (FK,UNIQ) |
|              | | member_id (FK) |
|              | | amount         |
|              | | is_paid        |
MEMBERS        +-----+
+-----+
| member_id (PK) |
| first_name     |
| last_name      |          STAFF
| email (UNIQUE) | +-----+
| type_id (FK)   | | staff_id (PK) |
| is_active      | | first_name     |
+-----+---+ | role           |
|              | +-----+

```

7. Sample Data Scripts

```

-- Publishers
INSERT INTO library.publishers (name, country, website) VALUES
('Penguin Random House', 'USA', 'https://penguinrandomhouse.com'),
('Oxford University Press', 'UK', 'https://oup.com'),
('O'Reilly Media', 'USA', 'https://oreilly.com'),
('Manning Publications', 'USA', 'https://manning.com'),
('HarperCollins', 'USA', 'https://harpercollins.com');

-- Genres
INSERT INTO library.genres (name) VALUES
('Fiction'),('Non-Fiction'),('Science Fiction'),
('Mystery'),('Technology'),('History'),('Biography'),('Self-Help');

INSERT INTO library.genres (name, parent_id) VALUES
('Database Technology', 5),('PostgreSQL', 10),('Software Engineering', 5);

-- Authors
INSERT INTO library.authors (first_name, last_name, nationality, birth_year) VALUES
('Frank', 'Herbert', 'American', 1920),
('Agatha', 'Christie', 'British', 1890),
('Brendan', 'Gregg', 'American', 1978),
('Craig', 'Lockhart', 'American', 1975),
('Yuval', 'Harari', 'Israeli', 1976);

-- Books
INSERT INTO library.books (isbn, title, publisher_id, genre_id, publication_year,
total_copies, available_copies, location_code)
VALUES

```

```

('9780441013593', 'Dune', 1, 3, 1965, 5, 5, 'SF-A-01'),
('9780007527526', 'Murder on the Orient Express', 5, 4, 1934, 3, 3, 'MY-B-03'),
('9781492077220', 'Systems Performance', 3, 5, 2020, 4, 4, 'TC-C-02'),
('9780192853523', 'Sapiens', 2, 2, 2011, 6, 6, 'NF-D-05'),
('9780061962127', 'The Great Gatsby', 5, 1, 1925, 4, 4, 'FI-A-07'),
('9781617295942', 'PostgreSQL: Up and Running', 4, 10, 2019, 3, 3, 'TC-E-01');

-- Book-Author links
INSERT INTO library.book_authors (book_id, author_id, role) VALUES
(1,1,'Author'),(2,2,'Author'),(3,3,'Author'),
(4,5,'Author'),(5,5,'Co-Author'),(6,4,'Author');

-- Staff
INSERT INTO library.staff (first_name, last_name, email, role) VALUES
('Alice', 'Morgan', 'alice.morgan@library.org', 'Senior Librarian'),
('Bob', 'Chen', 'bob.chen@library.org', 'Librarian'),
('Carla', 'Diaz', 'carla.diaz@library.org', 'Manager');

-- Members
INSERT INTO library.members (first_name, last_name, email, phone, type_id,
membership_date, expiry_date) VALUES
('James', 'Wilson', 'james.w@email.com', '555-0101', 2, '2024-01-15', '2025-
01-15'),
('Sarah', 'Johnson', 'sarah.j@email.com', '555-0102', 3, '2023-11-01', '2024-
11-01'),
('Michael', 'Brown', 'michael.b@email.com', '555-0103', 1, '2024-03-20', '2025-
03-20'),
('Emily', 'Davis', 'emily.d@email.com', '555-0104', 4, '2024-02-10', '2025-
02-10'),
('Robert', 'Martinez', 'robert.m@email.com', '555-0105', 2, '2023-08-05', '2024-
08-05');

-- Loans (some active, some returned, one overdue)
INSERT INTO library.loans (member_id, book_id, staff_id, loan_date, due_date,
return_date, status)
VALUES
(1, 1, 1, NOW() - INTERVAL '5 days', (NOW() - INTERVAL '5 days' + INTERVAL '14
days')::DATE, NULL, 'Active'),
(2, 3, 2, NOW() - INTERVAL '20 days', (NOW() - INTERVAL '20 days' + INTERVAL '14
days')::DATE, NULL, 'Overdue'),
(3, 2, 1, NOW() - INTERVAL '10 days', (NOW() - INTERVAL '10 days' + INTERVAL '14
days')::DATE, NOW() - INTERVAL '2 days', 'Returned'),
(4, 4, 2, NOW() - INTERVAL '3 days', (NOW() - INTERVAL '3 days' + INTERVAL '14
days')::DATE, NULL, 'Active'),
(1, 6, 1, NOW() - INTERVAL '30 days', (NOW() - INTERVAL '30 days' + INTERVAL '21
days')::DATE, NOW() - INTERVAL '5 days', 'Returned');

-- Fines
INSERT INTO library.fines (loan_id, member_id, amount, reason)
VALUES
(2, 2, 1.50, 'Late Return'),
(5, 1, 2.25, 'Late Return');

```



```
-- Reservations
INSERT INTO library.reservations (member_id, book_id, status) VALUES
  (3, 3, 'Pending'),
  (5, 1, 'Pending');
```

8. Core SQL Queries

```
-- Set schema
SET search_path = library, public;

-----
-- Q1: All books with author names and genre
-----
SELECT
  b.title,
  b.isbn,
  STRING_AGG(a.first_name || ' ' || a.last_name, ', ' ORDER BY a.last_name) AS
authors,
  g.name AS genre,
  b.available_copies,
  b.total_copies
FROM books b
LEFT JOIN book_authors ba ON b.book_id = ba.book_id
LEFT JOIN authors a      ON ba.author_id = a.author_id
LEFT JOIN genres g       ON b.genre_id = g.genre_id
GROUP BY b.book_id, b.title, b.isbn, g.name, b.available_copies, b.total_copies
ORDER BY b.title;

-----
-- Q2: Currently active loans with member and book info
-----
SELECT
  l.loan_id,
  m.first_name || ' ' || m.last_name AS member_name,
  b.title,
  l.loan_date::DATE AS borrowed_on,
  l.due_date,
  CURRENT_DATE - l.due_date AS days_overdue
FROM loans l
JOIN members m ON l.member_id = m.member_id
JOIN books b   ON l.book_id   = b.book_id
WHERE l.status = 'Active'
ORDER BY l.due_date;

-----
-- Q3: Overdue loans
-----
SELECT
  l.loan_id,
```

```

    m.first_name || ' ' || m.last_name AS member,
    m.email,
    b.title,
    l.due_date,
    CURRENT_DATE - l.due_date AS days_overdue,
    (CURRENT_DATE - l.due_date) * 0.25 AS estimated_fine
FROM loans l
JOIN members m ON l.member_id = m.member_id
JOIN books b ON l.book_id = b.book_id
WHERE l.return_date IS NULL AND l.due_date < CURRENT_DATE
ORDER BY days_overdue DESC;

```

```

-----
-- Q4: Member borrowing history (last 12 months)
-----

```

```

SELECT
    m.first_name || ' ' || m.last_name AS member,
    b.title,
    l.loan_date::DATE,
    l.due_date,
    l.return_date::DATE,
    l.status,
    COALESCE(f.amount, 0) AS fine_amount
FROM loans l
JOIN members m ON l.member_id = m.member_id
JOIN books b ON l.book_id = b.book_id
LEFT JOIN fines f ON l.loan_id = f.loan_id
WHERE m.member_id = 1
    AND l.loan_date >= NOW() - INTERVAL '1 year'
ORDER BY l.loan_date DESC;

```

```

-----
-- Q5: Top 5 most borrowed books
-----

```

```

SELECT
    b.title,
    COUNT(l.loan_id) AS times_borrowed,
    ROUND(AVG(EXTRACT(DAY FROM COALESCE(l.return_date, NOW()) - l.loan_date)), 1) AS
avg_loan_days
FROM loans l
JOIN books b ON l.book_id = b.book_id
GROUP BY b.book_id, b.title
ORDER BY times_borrowed DESC
LIMIT 5;

```

```

-----
-- Q6: Books currently checked out (not available)
-----

```

```

SELECT
    b.title,
    b.total_copies,
    b.available_copies,

```

```

        b.total_copies - b.available_copies AS copies_out,
        b.location_code
FROM books b
WHERE b.available_copies < b.total_copies
ORDER BY copies_out DESC;

-----
-- Q7: Members with unpaid fines
-----

SELECT
    m.first_name || ' ' || m.last_name AS member,
    m.email,
    COUNT(f.fine_id) AS unpaid_fines_count,
    SUM(f.amount) AS total_unpaid
FROM fines f
JOIN members m ON f.member_id = m.member_id
WHERE f.is_paid = FALSE
GROUP BY m.member_id, m.first_name, m.last_name, m.email
ORDER BY total_unpaid DESC;

-----
-- Q8: Books by genre with availability stats
-----

SELECT
    g.name AS genre,
    COUNT(b.book_id) AS book_titles,
    SUM(b.total_copies) AS total_copies,
    SUM(b.available_copies) AS available_copies,
    ROUND(100.0 * SUM(b.available_copies)
        / NULLIF(SUM(b.total_copies), 0), 1) AS availability_pct
FROM genres g
LEFT JOIN books b ON g.genre_id = b.genre_id
GROUP BY g.genre_id, g.name
ORDER BY book_titles DESC;

-----
-- Q9: Member activity ranked by loans (window function)
-----

SELECT
    m.first_name || ' ' || m.last_name AS member,
    mt.name AS membership_type,
    COUNT(l.loan_id) AS total_loans,
    RANK() OVER (ORDER BY COUNT(l.loan_id) DESC) AS rank
FROM members m
JOIN membership_types mt ON m.type_id = mt.type_id
LEFT JOIN loans l ON m.member_id = l.member_id
GROUP BY m.member_id, m.first_name, m.last_name, mt.name
ORDER BY total_loans DESC;

-----
-- Q10: Books never borrowed
-----

```

```

SELECT b.title, b.isbn, b.available_copies, p.name AS publisher
FROM books b
JOIN publishers p ON b.publisher_id = p.publisher_id
WHERE NOT EXISTS (
    SELECT 1 FROM loans l WHERE l.book_id = b.book_id
);

```

```

-- -----
-- Q11: Monthly loan statistics for current year
-- -----

```

```

SELECT
    DATE_TRUNC('month', loan_date)::DATE AS month,
    COUNT(*) AS total_loans,
    COUNT(*) FILTER (WHERE status = 'Returned') AS returned,
    COUNT(*) FILTER (WHERE status = 'Active') AS active,
    COUNT(*) FILTER (WHERE status = 'Overdue') AS overdue
FROM loans
WHERE loan_date >= DATE_TRUNC('year', NOW())
GROUP BY 1
ORDER BY 1;

```

```

-- -----
-- Q12: Authors with their book count and average availability
-- -----

```

```

SELECT
    a.first_name || ' ' || a.last_name AS author,
    COUNT(DISTINCT ba.book_id) AS books_in_library,
    SUM(b.total_copies) AS total_copies,
    ROUND(AVG(b.available_copies), 1) AS avg_available
FROM authors a
JOIN book_authors ba ON a.author_id = ba.author_id
JOIN books b ON ba.book_id = b.book_id
GROUP BY a.author_id, a.first_name, a.last_name
ORDER BY books_in_library DESC;

```

```

-- -----
-- Q13: Members approaching membership expiry (next 30 days)
-- -----

```

```

SELECT
    first_name || ' ' || last_name AS member,
    email,
    expiry_date,
    expiry_date - CURRENT_DATE AS days_until_expiry
FROM members
WHERE is_active = TRUE
    AND expiry_date BETWEEN CURRENT_DATE AND CURRENT_DATE + 30
ORDER BY expiry_date;

```

```

-- -----
-- Q14: Full-text search for books
-- -----

```

```

SELECT

```

```

        title,
        isbn,
        ts_rank(to_tsvector('english', title), query) AS rank
FROM books,
        to_tsquery('english', 'systems & performance') AS query
WHERE to_tsvector('english', title) @@ query
ORDER BY rank DESC;

-- -----
-- Q15: Loans processed by each staff member
-- -----

SELECT
    s.first_name || ' ' || s.last_name AS staff,
    s.role,
    COUNT(l.loan_id) AS loans_processed,
    COUNT(l.loan_id) FILTER (WHERE l.return_date IS NOT NULL) AS returns_processed
FROM staff s
LEFT JOIN loans l ON s.staff_id = l.staff_id
GROUP BY s.staff_id, s.first_name, s.last_name, s.role
ORDER BY loans_processed DESC;

-- -----
-- Q16: Books with pending reservations
-- -----

SELECT
    b.title,
    b.available_copies,
    COUNT(r.reservation_id) AS pending_reservations,
    MIN(r.reserved_at)::DATE AS oldest_reservation
FROM reservations r
JOIN books b ON r.book_id = b.book_id
WHERE r.status = 'Pending'
GROUP BY b.book_id, b.title, b.available_copies
ORDER BY pending_reservations DESC;

-- -----
-- Q17: Running total of fines by member (window function)
-- -----

SELECT
    m.first_name || ' ' || m.last_name AS member,
    f.issued_at::DATE AS fine_date,
    f.amount,
    f.reason,
    SUM(f.amount) OVER (
        PARTITION BY f.member_id
        ORDER BY f.issued_at
        ROWS UNBOUNDED PRECEDING
    ) AS running_total_fines
FROM fines f
JOIN members m ON f.member_id = m.member_id
ORDER BY f.member_id, f.issued_at;

```

```
-- -----  
-- Q18: Average loan duration by genre  
-- -----
```

```
SELECT  
    g.name AS genre,  
    COUNT(l.loan_id) AS completed_loans,  
    ROUND(AVG(  
        EXTRACT(DAY FROM l.return_date - l.loan_date)  
    ), 1) AS avg_days_to_return  
FROM loans l  
JOIN books b ON l.book_id = b.book_id  
JOIN genres g ON b.genre_id = g.genre_id  
WHERE l.return_date IS NOT NULL  
GROUP BY g.genre_id, g.name  
HAVING COUNT(l.loan_id) > 0  
ORDER BY avg_days_to_return;
```

```
-- -----  
-- Q19: CTE - Members who never paid a fine but have overdue books  
-- -----
```

```
WITH overdue_members AS (  
    SELECT DISTINCT l.member_id  
    FROM loans l  
    WHERE l.return_date IS NULL AND l.due_date < CURRENT_DATE  
)  
,  
fine_paid_members AS (  
    SELECT DISTINCT member_id  
    FROM fines  
    WHERE is_paid = TRUE  
)  
SELECT m.first_name || ' ' || m.last_name AS member, m.email  
FROM members m  
JOIN overdue_members om ON m.member_id = om.member_id  
WHERE m.member_id NOT IN (SELECT member_id FROM fine_paid_members);
```

```
-- -----  
-- Q20: Inventory status report  
-- -----
```

```
SELECT  
    b.title,  
    p.name AS publisher,  
    g.name AS genre,  
    b.total_copies,  
    b.available_copies,  
    (SELECT COUNT(*) FROM loans l  
     WHERE l.book_id = b.book_id AND l.status = 'Active') AS active_loans,  
    (SELECT COUNT(*) FROM reservations r  
     WHERE r.book_id = b.book_id AND r.status = 'Pending') AS pending_reservations  
FROM books b  
JOIN publishers p ON b.publisher_id = p.publisher_id
```

```
LEFT JOIN genres g ON b.genre_id = g.genre_id
ORDER BY b.title;
```

9. Stored Procedures / Functions

```
-- -----
-- FUNCTION 1: Checkout a book
-- Checks member validity, book availability, current loan count
-- Updates available_copies and inserts loan record
-- -----

CREATE OR REPLACE FUNCTION library.checkout_book(
    p_member_id INTEGER,
    p_book_id   INTEGER,
    p_staff_id  INTEGER DEFAULT NULL
)
RETURNS TABLE (loan_id INTEGER, due_date DATE, message TEXT)
LANGUAGE plpgsql AS $$
DECLARE
    v_member      library.members%ROWTYPE;
    v_book        library.books%ROWTYPE;
    v_membership   library.membership_types%ROWTYPE;
    v_active_loans INTEGER;
    v_new_loan_id  INTEGER;
    v_due_date     DATE;
BEGIN
    -- Fetch and validate member
    SELECT * INTO v_member FROM library.members WHERE member_id = p_member_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Member ID % not found', p_member_id;
    END IF;
    IF NOT v_member.is_active THEN
        RAISE EXCEPTION 'Member % is not active', p_member_id;
    END IF;
    IF v_member.expiry_date < CURRENT_DATE THEN
        RAISE EXCEPTION 'Membership for member % has expired', p_member_id;
    END IF;

    -- Fetch membership type limits
    SELECT * INTO v_membership FROM library.membership_types WHERE type_id =
v_member.type_id;

    -- Count active loans for member
    SELECT COUNT(*) INTO v_active_loans
    FROM library.loans
    WHERE member_id = p_member_id AND status = 'Active';

    IF v_active_loans >= v_membership.max_books THEN
        RAISE EXCEPTION 'Member % has reached maximum loan limit of %',
            p_member_id, v_membership.max_books;
    END IF;
```

```

-- Fetch and validate book
SELECT * INTO v_book FROM library.books WHERE book_id = p_book_id FOR UPDATE;
IF NOT FOUND THEN
    RAISE EXCEPTION 'Book ID % not found', p_book_id;
END IF;
IF v_book.available_copies <= 0 THEN
    RAISE EXCEPTION 'No available copies for book ID %', p_book_id;
END IF;

-- Check for unpaid fines exceeding $5
IF (SELECT COALESCE(SUM(amount),0) FROM library.fines
    WHERE member_id = p_member_id AND is_paid = FALSE) > 5.00 THEN
    RAISE EXCEPTION 'Member % has outstanding fines over $5.00', p_member_id;
END IF;

-- Calculate due date
v_due_date := CURRENT_DATE + v_membership.loan_days;

-- Insert loan
INSERT INTO library.loans (member_id, book_id, staff_id, loan_date, due_date,
status)
VALUES (p_member_id, p_book_id, p_staff_id, NOW(), v_due_date, 'Active')
RETURNING library.loans.loan_id INTO v_new_loan_id;

-- Decrement available copies
UPDATE library.books
SET available_copies = available_copies - 1,
    updated_at = NOW()
WHERE book_id = p_book_id;

RETURN QUERY SELECT v_new_loan_id, v_due_date, 'Checkout successful'::TEXT;
END;
$$;

-----
-- FUNCTION 2: Return a book and calculate fines
-----

CREATE OR REPLACE FUNCTION library.return_book(
    p_loan_id INTEGER,
    p_staff_id INTEGER DEFAULT NULL
)
RETURNS TABLE (fine_amount NUMERIC, message TEXT)
LANGUAGE plpgsql AS $$
DECLARE
    v_loan      library.loans%ROWTYPE;
    v_fine_amt  NUMERIC(10,2) := 0.00;
    v_days_late INTEGER;
BEGIN
    SELECT * INTO v_loan FROM library.loans WHERE loan_id = p_loan_id FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Loan ID % not found', p_loan_id;
    
```



```

END IF;
IF v_loan.status = 'Returned' THEN
    RAISE EXCEPTION 'Loan % is already returned', p_loan_id;
END IF;

-- Calculate overdue fine
v_days_late := GREATEST(0, CURRENT_DATE - v_loan.due_date);
v_fine_amt := v_days_late * 0.25;

-- Update loan record
UPDATE library.loans
SET return_date = NOW(),
    status      = 'Returned'
WHERE loan_id = p_loan_id;

-- Increment available copies
UPDATE library.books
SET available_copies = available_copies + 1,
    updated_at = NOW()
WHERE book_id = v_loan.book_id;

-- Issue fine if applicable
IF v_fine_amt > 0 THEN
    INSERT INTO library.fines (loan_id, member_id, amount, reason)
    VALUES (p_loan_id, v_loan.member_id, v_fine_amt, 'Late Return');

    UPDATE library.members
    SET total_fines = total_fines + v_fine_amt
    WHERE member_id = v_loan.member_id;
END IF;

RETURN QUERY SELECT
    v_fine_amt,
    CASE WHEN v_fine_amt > 0
        THEN format('Book returned. Fine of $%s applied for %s overdue days.',
                    v_fine_amt, v_days_late)
        ELSE 'Book returned on time. No fine.'
    END;
END;
$$;

-----
-- FUNCTION 3: Get member summary
-----

CREATE OR REPLACE FUNCTION library.get_member_summary(p_member_id INTEGER)
RETURNS TABLE (
    member_name      TEXT,
    membership_type  TEXT,
    active_loans     BIGINT,
    total_loans      BIGINT,
    unpaid_fines     NUMERIC,
    membership_status TEXT

```

```

)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        m.first_name || ' ' || m.last_name,
        mt.name,
        COUNT(l.loan_id) FILTER (WHERE l.status = 'Active'),
        COUNT(l.loan_id),
        COALESCE(SUM(f.amount) FILTER (WHERE f.is_paid = FALSE), 0.00),
        CASE
            WHEN m.expiry_date < CURRENT_DATE THEN 'Expired'
            WHEN NOT m.is_active THEN 'Inactive'
            ELSE 'Active'
        END
    FROM library.members m
    JOIN library.membership_types mt ON m.type_id = mt.type_id
    LEFT JOIN library.loans l ON m.member_id = l.member_id
    LEFT JOIN library.fines f ON m.member_id = f.member_id
    WHERE m.member_id = p_member_id
    GROUP BY m.member_id, m.first_name, m.last_name, mt.name,
             m.expiry_date, m.is_active;
END;
$$;

```

10. Triggers

```

-- -----
-- TRIGGER 1: Auto-update loan status to 'Overdue'
-- Runs daily via pg_cron or on-read; here demonstrated
-- as a trigger on UPDATE of loans table
-- -----

CREATE OR REPLACE FUNCTION library.trg_update_overdue_status()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.return_date IS NULL AND NEW.due_date < CURRENT_DATE THEN
        NEW.status := 'Overdue';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_loans_overdue
BEFORE INSERT OR UPDATE ON library.loans
FOR EACH ROW
EXECUTE FUNCTION library.trg_update_overdue_status();

-- -----
-- TRIGGER 2: Prevent borrowing more than membership limit
-- (Defense-in-depth alongside application logic)

```

```

-----
CREATE OR REPLACE FUNCTION library.trg_enforce_loan_limit()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_active_count INTEGER;
    v_max_books    SMALLINT;
BEGIN
    SELECT COUNT(*) INTO v_active_count
    FROM library.loans
    WHERE member_id = NEW.member_id AND status = 'Active';

    SELECT mt.max_books INTO v_max_books
    FROM library.members m
    JOIN library.membership_types mt ON m.type_id = mt.type_id
    WHERE m.member_id = NEW.member_id;

    IF v_active_count >= v_max_books THEN
        RAISE EXCEPTION 'Loan limit exceeded for member %. Max allowed: %',
            NEW.member_id, v_max_books;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_loans_limit
BEFORE INSERT ON library.loans
FOR EACH ROW
EXECUTE FUNCTION library.trg_enforce_loan_limit();

```

11. Performance Optimization

Index	Table	Columns	Reason
idx_books_title (GIN)	books	to_tsvector(title)	Full-text search on book titles
idx_books_available	books	available_copies WHERE > 0	Partial index for availability checks
idx_loans_status	loans	status WHERE IN (Active,Overdue)	Partial index — most queries filter by active loans
idx_loans_due_date	loans	due_date WHERE return_date IS NULL	Overdue query, filtered partial
idx_fines_unpaid	fines	member_id WHERE is_paid=FALSE	Member outstanding balance lookups
idx_members_active	members	is_active WHERE TRUE	Only active members queried frequently

EXPLAIN ANALYZE Example:

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT b.title, m.email
FROM loans l
JOIN books b ON l.book_id = b.book_id
JOIN members m ON l.member_id = m.member_id
WHERE l.status = 'Overdue'
      AND l.return_date IS NULL;
```

12. Extensions Used

```
-- Enable pg_trgm for fuzzy title search
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Create trigram index for LIKE/ILIKE search
CREATE INDEX idx_books_title_trgm
  ON library.books
  USING GIN (title gin_trgm_ops);

-- Fuzzy search example
SELECT title FROM library.books
WHERE title % 'Sapins' -- typo tolerance
ORDER BY similarity(title, 'Sapins') DESC;

-- Enable pgcrypto for member ID hashing (if needed)
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Enable pg_stat_statements for query monitoring
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

13. Testing Guide

```
-- TEST 1: Successful checkout
SELECT * FROM library.checkout_book(1, 1, 1); -- Should succeed

-- TEST 2: Checkout with no available copies
UPDATE library.books SET available_copies = 0 WHERE book_id = 2;
SELECT * FROM library.checkout_book(1, 2, 1); -- Should raise error

-- TEST 3: Return a book on time
SELECT * FROM library.return_book(1, 2); -- No fine expected

-- TEST 4: Return an overdue book
UPDATE library.loans SET due_date = CURRENT_DATE - 6 WHERE loan_id = 1;
SELECT * FROM library.return_book(1, 2); -- Fine = 6 * 0.25 = $1.50

-- TEST 5: Exceed loan limit
-- Give member 3 a Standard membership (max 5 books)
```

```
-- Insert 5 active loans, then try a 6th:
SELECT * FROM library.checkout_book(3, 5, 1); -- Should fail on 6th

-- TEST 6: Verify member summary
SELECT * FROM library.get_member_summary(2);

-- TEST 7: Full text search
SELECT title FROM library.books
WHERE to_tsvector('english', title) @@ to_tsquery('english', 'performance');

-- TEST 8: Overdue trigger
INSERT INTO library.loans (member_id, book_id, loan_date, due_date)
VALUES (5, 5, NOW() - INTERVAL '20 days', CURRENT_DATE - 5);
SELECT status FROM library.loans WHERE member_id = 5 ORDER BY loan_id DESC LIMIT 1;
-- Should be 'Overdue'
```

14. Extension Challenges

- Digital Catalog:** Add `ebook_url` and `audio_url` columns to `books`. Track digital loans separately with no copy limits. Implement a `digital_loans` table with concurrent access limits.
- Inter-Library Loans:** Add a `partner_libraries` table and support loans from external libraries. Track source library in the `loans` table and add routing logic.
- Recommendation Engine:** Build a `member_preferences` table derived from loan history. Write a function that recommends books based on genre frequency and co-borrowing patterns.
- Notifications System:** Add an `email_queue` table. Write a trigger that enqueues notifications when books go overdue or reservations become available. Integrate with `pg_cron` to flush the queue hourly.
- Analytics Dashboard:** Create materialized views for monthly stats, top borrowers, and genre trends. Refresh them nightly with `pg_cron` and expose them as a reporting schema.

15. What You Learned

Skill	Demonstrated By
Schema normalization (3NF)	8-table design with no redundancy
Constraints and data integrity	CHECK, UNIQUE, FK constraints throughout
PL/pgSQL programming	<code>checkout_book</code> , <code>return_book</code> , <code>get_member_summary</code>
Trigger-based automation	Overdue status, loan limit enforcement
Window functions	Member ranking, running fine totals
Full-text search	GIN index on title, <code>to_tsvector</code> queries
Partial indexes	Targeted indexes on active/unpaid rows
CTEs	Multi-step analytical queries

Aggregate reporting	Genre stats, staff performance, monthly trends
Fine and transaction logic	Business rules encoded in SQL